UNIT III

Static Testing: Inspections, Structured Walkthroughs, Technical Reviews.

Validation activities: Unit testing, Integration Testing, Function testing, system testing, acceptance testing.

Regression testing: Progressives Vs regressive testing, Regression test ability, Objectives of regression testing, Regression testing types, Regression testing techniques.

Validation Activities

1.Validation activities: Unit testing, Integration Testing, Function testing, system testing, acceptance testing.

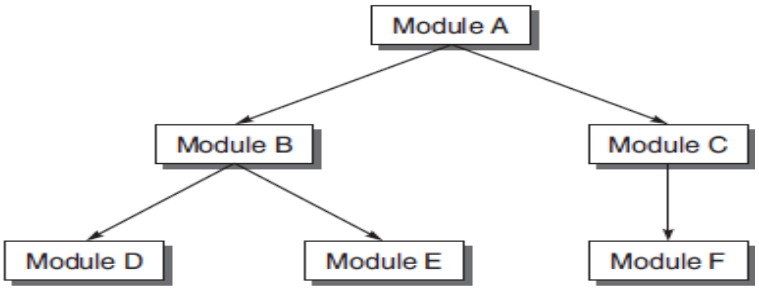
Unit Testing

- A unit is the smallest testable part of an application like functions, classes, procedures, interfaces.
- Unit testing is a method by which individual units of source code are tested to determine if they are fit for use.
- Unit tests are basically written and executed by software developers to make sure that code meets its design and requirements and behaves as expected.
- The goal of unit testing is to isolate each part of the program and test that the individual parts are working correctly.
- This means that for any function or procedure when a set of inputs are given then it should return the proper values.
- Software is divided into modules but a module is not an isolated entity because it gets some inputs from another module or the module is calling some other module.
- It means that a module is not independent and cannot be tested in isolation.
- While testing the module, all its interfaces must be simulated (imitation of some other) if the interfaced modules are not ready at the time of testing the module under consideration.
- Below is the types of interface modules which must be simulated, if required to test a module.

Drivers

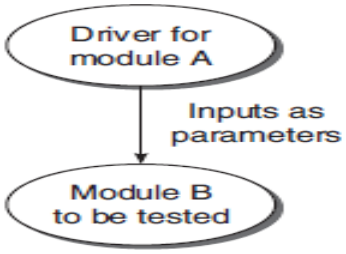
- Drivers are also kind of dummy modules which are known as "calling programs", which is used when main programs are under construction.
- Suppose a module is to be tested, where in some inputs are to be received from another module.
- However this module which passes inputs to the module to be tested is not ready and under development. In such a situation, we need to simulate the inputs required in the module to be tested.

- This module where the required inputs for the module under test are simulated for the purpose of module or unit testing is known as **driver module**.
- For example, the design hierarchy of the modules, as shown in below figure



Design hierarchy of an example system

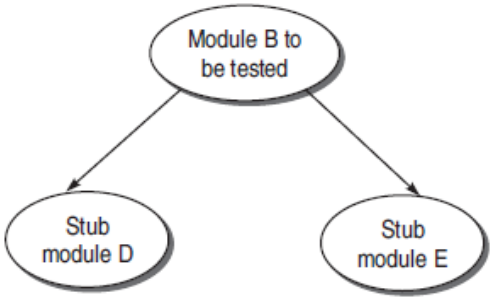
- Suppose module B is under test. In the hierarchy, module A is a super-ordinate of module B. Suppose module A is not ready and B has to be unit tested. In this case, module B needs inputs from module A.
- Therefore, a driver module is needed which will simulate module A in the sense that it passes the required inputs to module B and acts as a main program for module B in which its being called, as shown in below figure.



Driver Module for module A

Stubs

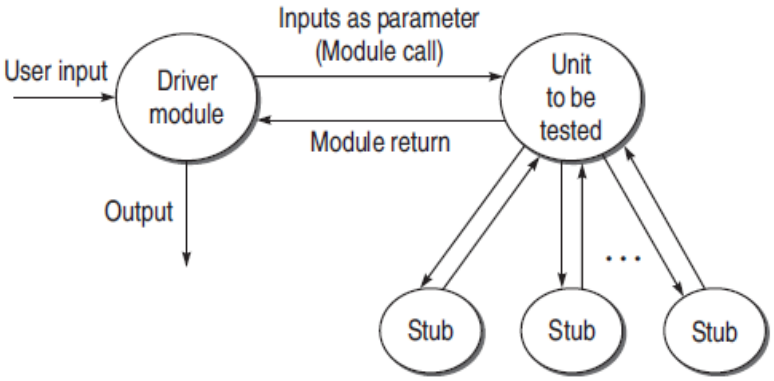
- Stubs are dummy modules which are known as "called programs" which is used when sub programs are under construction.
- The module under testing may also call some other module which is not ready at the time of testing. Therefore, these modules need to be simulated for testing.
- In most cases, dummy modules instead of actual modules, which are not ready, are prepared for these subordinate modules are called **Stubs.**
- For example, Module B under test needs to call module D and module E. But they are not ready. So there must be some skeletal structure in their place so that they act as dummy modules in place of the actual modules.
- Therefore, stubs are designed for module D and module E, as shown in below figure:



Stubs

Benefits of Designing Stubs and Drivers

- Stubs allow the programmer to call a method in the code being developed, even if the method does not have the desired behavior yet.
- By using stubs and drivers effectively, we can cut down our total debugging and testing time by testing small parts of a program individually, helping us to narrow down problems before they expand.
- Stubs and Drivers can also be an effective tool for demonstrating progress in a business environment.



Drivers and Stubs

Example

Consider the following program:

```
main()
{
        int a,b,c,sum,diff,mul;
        scanf("%d %d %d", &a, &b, &c);
        sum = calsum(a,b,c);
        diff = caldiff(a,b,c);
        mul = calmul(a,b,c);
        printf("%d %d %d", sum, diff, mul);
}
calsum(int x, int y, int z)
{
        int d;

        d = x + y + z;
        return(d);
```

}

(a) Suppose main() module is not ready for the testing of calsum() module. Design a driver module for main().

(b) Modules caldiff() and calmul() are not ready when called in main().Design stubs for these two modules.

Solution

(a) **Driver for** main() **module:**

```
main()
{
        int a, b, c, sum;
        scanf("%d %d %d", &a, &b, &c);
        sum = calsum(a,b,c);
        printf("The output from calsum module is %d", sum);
}
```

(b) **Stub for** caldiff() **Module**

```
caldiff(int x, int y, int z)
{
        printf("Difference calculating module");
        return 0;
}
```

Stub for calmul() **Module**

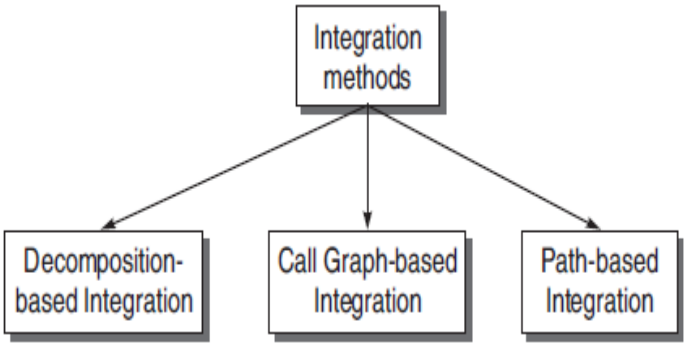
```
calmul(int x, int y, int z)
{
        printf("Multiplication calculation module");
        return 0;
}
```

2.Integration Testing

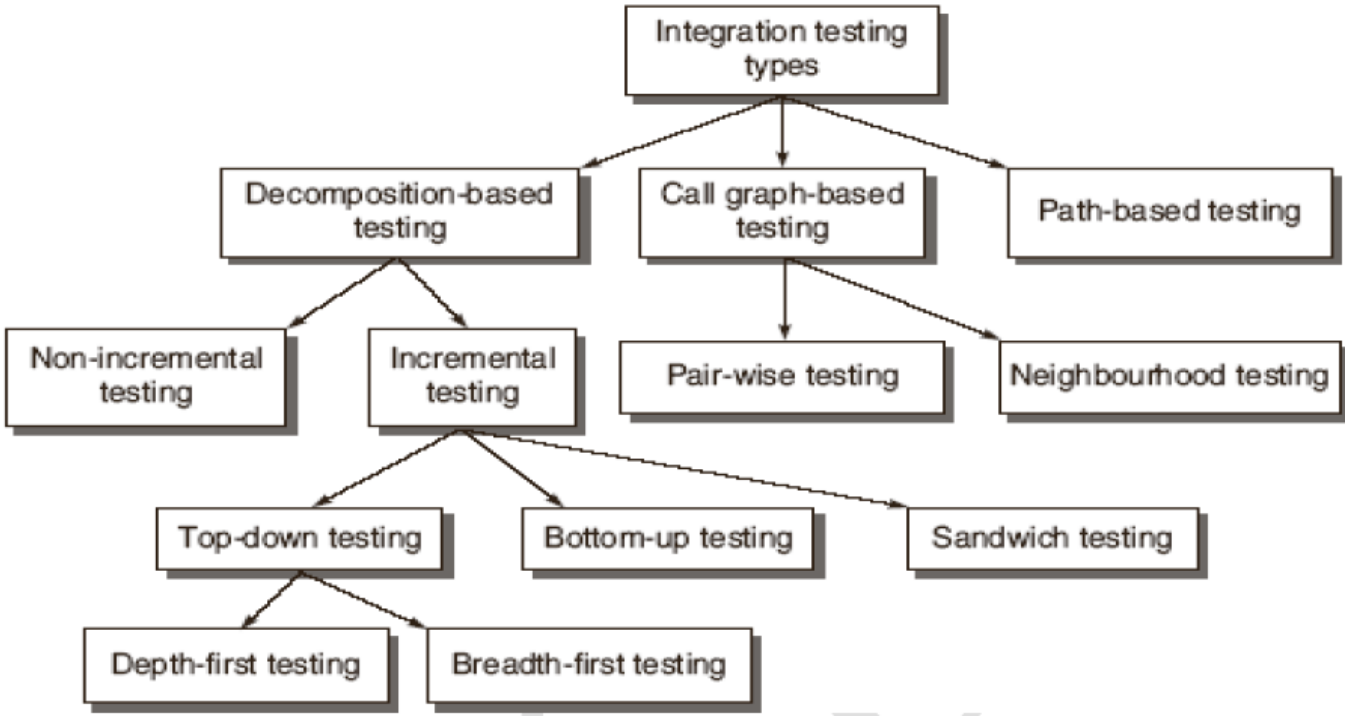
- Once all the individual units are created and tested, combining those "Unit Tested" modules and start doing the integrated testing.
- Therefore, the meaning of Integration testing is Integrate/combine the unit tested module one by one and test the behavior as a combined unit.
- The main function or goal of Integration testing is to test the interfaces between the units/modules.
- The individual modules are first tested in isolation. Once the modules are unit tested, they are integrated one by one, till all the modules are integrated, to check the combinational behavior, and validate whether the requirements are implemented correctly or not.
- Integration Testing is necessary for the following reasons:
    - It exposes inconsistency between the modules such as improper call or return sequences.

- Data can be lost across an interface.
- One module when combined with another module may not give the desired result.
- Data types and their valid ranges may mismatch between the modules.

- There are three approaches for integration testing:

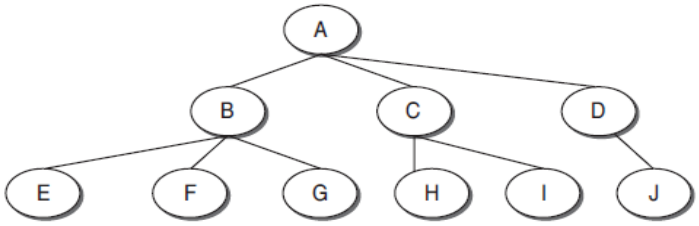


Integration Methods



2.1.Decomposition Based Integration

- In this strategy, we do the decomposition based on the functional characteristics of the system.
- A functional characteristic is defined by what the module does, that is, actions or activities performed by the module.
- The functional decomposition is shown as a tree in the below figure:

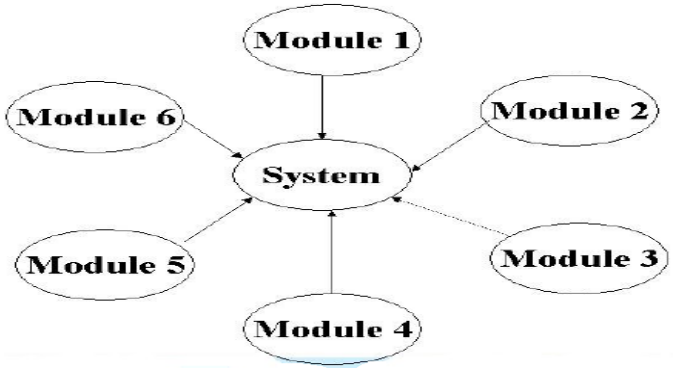


Decomposition Tree

- In the above tree designed for decomposition-based integration, the nodes represent the modules present in the system and the links/edges between the two modules represent the calling sequence. The nodes on the last level in the tree are leaf nodes.
- Module A is linked to three subordinate modules, B, C, and D. It means that module A calls modules, B, C, and D.
- There are four approaches for this strategy:
    - Non Incremental Integration Testing / Big bang integration
    - Incremental Integration Testing
        - Top-Down Integration
        - Bottom-up Integration
    - Practical Approach for Integration Testing / Sandwich integration.

Non-Incremental Integration Testing/ Big bang integration

- This is one of the easiest approaches to apply in integration testing.
- Here we treat the whole system as a subsystem and test it in a single test phase.
- Normally this means simply integrating all the modules, compile them all at once and then test the resulting system.
- This approach requires little resources to execute as we do not need to identify critical components (like interactions, paths between the modules) nor require extra coding for the "dummy modules".
- This approach may be used for very small systems, however it is still not recommended because it is not systematic.
- In larger systems, the low resources requirement in executing this testing is easily offset by the resources required to locate the problem when it occurs.



Big bang integration Testing

Big Bang integration has the following characteristics:

1. Considers the whole system as a subsystem
2. Tests all the modules in a single test session

3. Only one integration testing session

Advantages

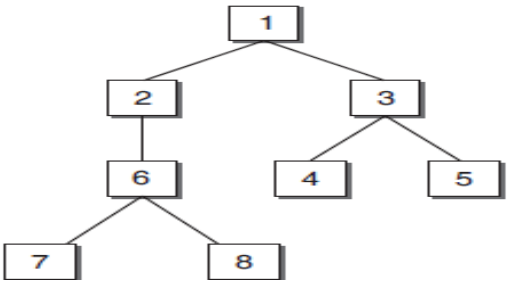
- Low resources requirement
- Does not require extra coding

Disadvantages

- Not systematic
- Hard to locate problems
- Hard to create test cases

Incremental Integration Testing

- In Incremental integration testing, the developers integrate the modules one by one using stubs or drivers to uncover the defects.
- This approach is known as incremental integration testing.
- Incremental integration testing is beneficial for the following reasons:
    1. Incremental approach does not require many drivers and stubs.
    2. Interfacing errors are uncovered earlier.
    3. It is easy to localize the errors since modules are combined one by one.
    4. Incremental testing is a more thorough testing. Consider the below example If we are testing module 6, then although module 1 and 2 have been tested previously, they will get the chance to be tested again and again.



- A disadvantage is that it can be time-consuming since stubs and drivers have to be developed for performing these tests.

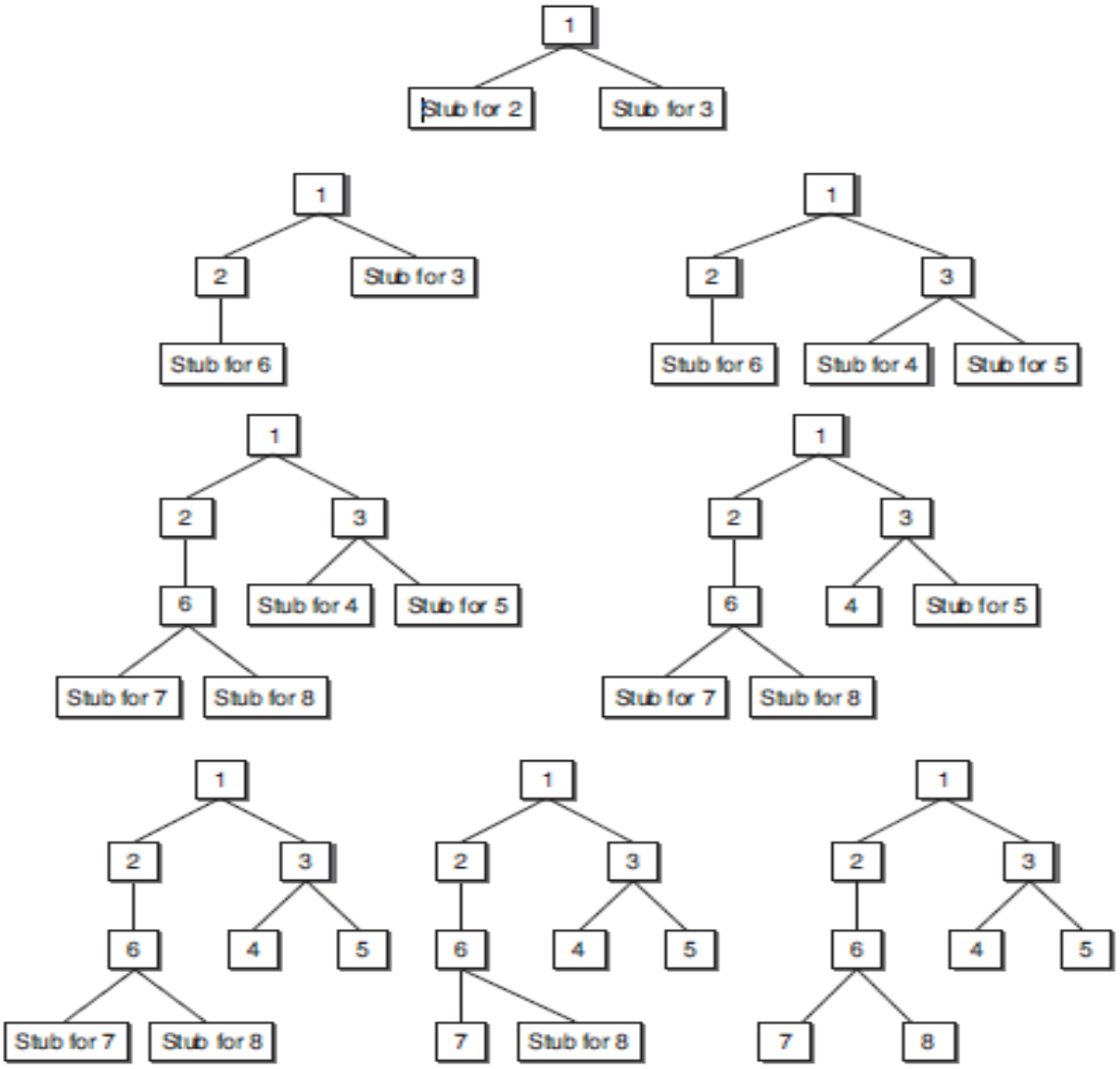
Types of Incremental integration testing

1. Top Down integration testing
2. Bottom-up integration testing

Top down Integration Testing:

- In top-down integration, we start at the target node at root of the functional decomposition tree and work toward the leaves.
- Stubs are used to replace the children nodes attached to the target node.

- A test phase consists in replacing one of the stub modules with the real code and test the resulting subsystem.
- If no problem is encountered then we do the next test phase.
- If all the children were replaced by real code and meet the requirements then we move down to the next level.
- Now we can replace the higher level tested modules with real code and continue the integration testing.
- For top-down integration the number of integration testing sessions is: **nodes−leaves +edges.** The top-down integration is shown below.



Modules subordinate to the top module are integrated in the following two ways:

Depth First Integration:

- In this type, all modules on a major control path of the design hierarchy are integrated first.
- In the above example modules 1, 2, 6, 7,8 will be integrated first. Next, modules 1, 3, 4,5 will be integrated.

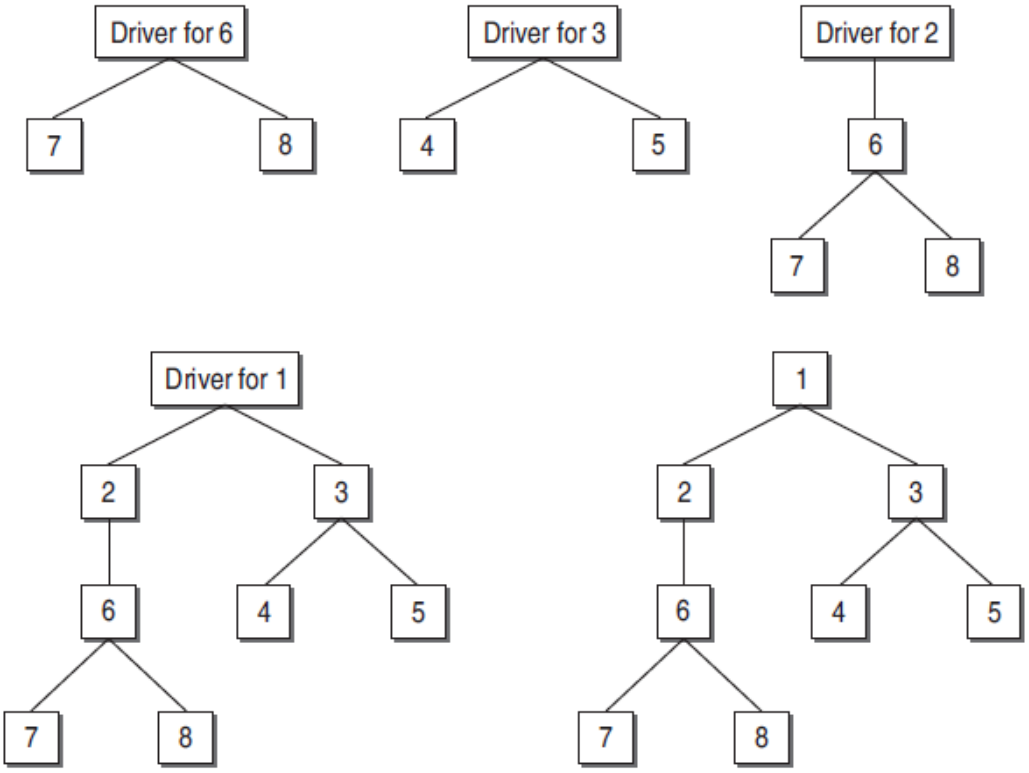
Breadth first integration:

- In this type, all modules directly subordinate at each level, moving across the design hierarchy horizontally, are integrated first.

- In the above example modules 2 and 3 will be integrated first. Next, modules 6, 4, and 5 will be integrated. Modules 7 and 8 will be integrated last.

Bottom-up Integration Testing

- Bottom-up integration starts at the opposite end of the functional decomposition tree, instead of starting at the main program we start at the lower-level implementation of the software.
- By moving in an opposite direction, the parent nodes are replaced by throw-away codes instead of the children.
- These throw-away codes are also known as drivers.
- The steps in bottom-up integration are as follows:
    1. Start with the lowest level modules in the design hierarchy. These are the modules from which no other module is being called.
    2. Look for the super-ordinate module which calls the module selected in step 1. Design the driver module for this super-ordinate module.
    3. Test the module selected in step 1 with the driver designed in step 2.
    4. The next module to be tested is any module whose subordinate modules have all been tested.
    5. Repeat steps 2 to 5 and move up in the design hierarchy.
    6. Whenever, the actual modules are available, replace stubs and drivers with the actual one and test again.
- Bottom-up integration is shown below:



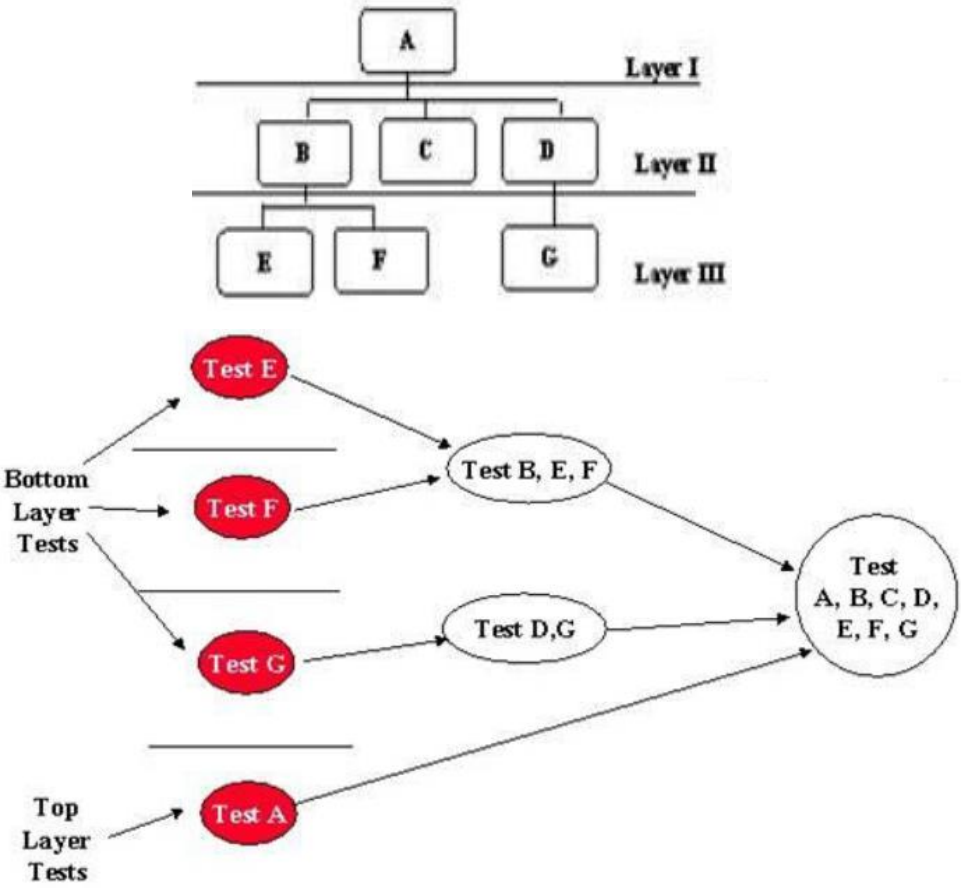
Comparison between Top-Down and Bottom-Up Integration Testing

Issue	Top-Down Testing	Bottom-Up Testing
Architectural Design	It discovers errors in high-level design, thus detects errors at an early stage.	High-level design is validated at a later stage.
System Demonstration	Since we integrate the modules from top to bottom, the high-level design slowly expands as a working system. Therefore, feasibility of the system can be demonstrated to the top management.	It may not be possible to show the feasibility of the design. However, if some modules are already built as reusable components, then it may be possible to produce some kind of demonstration.
Test Implementation	*(nodes – 1)* stubs are required for the subordinate modules.	*(nodes – leaves)* test drivers are required for super-ordinate modules to test the lower-level modules.

Practical Approach for Integration Testing/ Sandwich integration

- Sandwich integration combines top-down integration and bottom-up integration.
- The main concept is to maximize the advantages of top-down and bottom-up and minimizing their weaknesses.
- Sandwich integration uses a mixed-up approach where we use stubs at the higher level of the tree and drivers at the lower level.

- The testing direction starts from both side of tree and converges to the centre, thus the term sandwich.
- This will allow us to test both the top and bottom layers in parallel and decrease the number of stubs and drivers required in integration testing.



Advantages

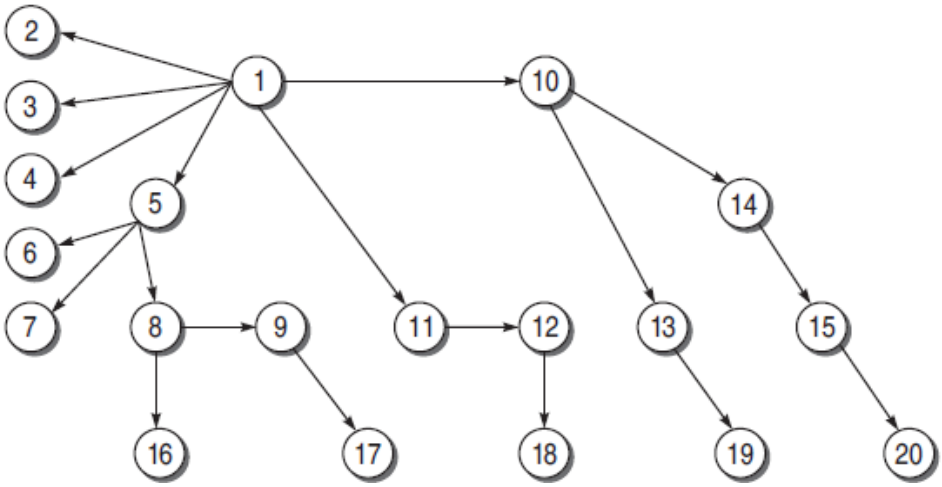
- Top and bottom layers can be done in parallel
- Less stubs and drivers needed
- Easy to construct test cases
- Integration is done as soon a component is implemented

Disadvantages

- Still requires throw-away code programming
- Partial big bang integration
- Hard to isolate problems

2.2 Call Graph-Based Integration

- A call graph is a directed graph, where the nodes are either modules or units, and a directed edge from one node to another node means one module has called another module.
- The call graph can be captured in a matrix form which is known as the adjacency matrix.



Example call graph

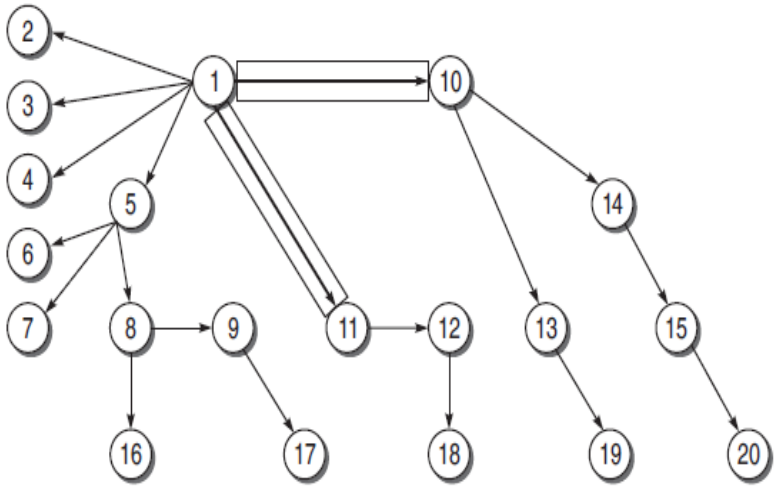
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	x	x	x	x					x	x									
2																			
3																			
4																			
5					x	x	x												
6																			
7																			
8								x							x				
9																x			
10												x	x						
11											x								
12																	x		
13																		x	
14														x					
15																			x
16																			
17																			
18																			
19																			
20																			

Adjacency matrix

- The idea behind using a call graph for integration testing is to avoid the efforts made in developing the stubs and drivers.
- If we know the calling sequence, and if we wait for the called or calling function, if not ready, then call graph-based integration can be used.
- There are two types of integration testing based on call graph
    1. Pair wise Integration
    2. Neighborhood Integration

Pair-wise Integration

- Consider only one pair of calling and called modules, and then we can make a set of pairs for all such modules for pairs 1–10 and 1–11.
- The resulting set will be the total test sessions which will be equal to the sum of all edges in the call graph.
- For example, in the call graph shown in below figure the number of test sessions is 19 which is equal to the number of edges in the call graph.



Pair-wise integration

Neighbourhood Integration

- There is not much reduction in the total number of test sessions in pair-wise integration.
- If we consider the neighbourhoods of a node in the call graph, then the number of test sessions may reduce.
- The neighbourhood for a node is the immediate predecessor as well as the immediate successor nodes.
- The neighbourhood of a node, can be defi ned as the set of nodes that are one edge away from the given node.
- The neighbourhoods of each node for the call above graph is shown below:

Node	Neighbourhoods	
	Predecessors	Successors
1	-----	2,3,4,5,10,11
5	1	6,7,8
8	5	9,16
9	8	17
10	1	13,14
11	1	12
12	11	18
13	10	19
14	10	15
15	14	20

- The total test sessions in neighbourhood integration can be calculated as:

Neighbourhoods = nodes – sink nodes

= 20 – 10

= 10

Where sink node is an instruction in a module at which the execution terminates.

2.3 Path-Based Integration

- In a call graph, when a module or unit executes, some path of source instructions is executed.
- in that path execution, there may be a call to another unit at that point, the control is transferred from the calling unit to the called unit.
- This must be tested at the time of integration. It can be done with the help of path-based integration.
- We need to understand the following definitions for path-based integration:

Source node

- It is an instruction in the module at which the execution starts or resumes.
- The nodes where the control is being transferred after calling the module are also source nodes.

Sink node

- It is an instruction in a module at which the execution terminates.
- The nodes from which the control is transferred is known as sink nodes.

Module execution path (MEP)

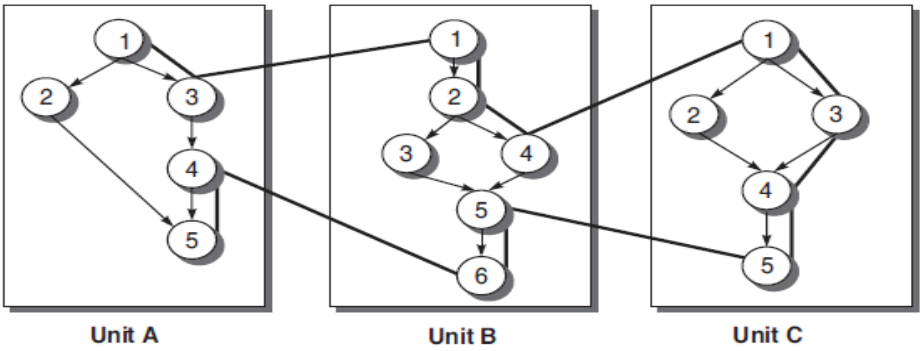
- It is a path consisting of a set of executable statements like in a flow graph.
- A sequence of statements within a module that begins with a source node, Ends with a sink node with no intervening sink nodes.

Message

- When the control from one unit is transferred to another unit, then the programming language mechanism used to do this is known as a *message*.
- For example, when there is a function call, then it is a message from one unit.

MM-path

- A module to module path.
- It is an interleaved sequence of module execution paths and messages which are used to describe sequences of module execution paths that include transfers of control among separate units.
- MM-paths always represent feasible execution paths, and these paths cross unit boundaries.



MM-path

	Source Nodes	Sink Nodes	MEPs
Unit A	1,4	3,5	MEP(A,1) = <1,2,5> MEP(A,2) = <1,3> MEP(A,3) = <4,5>
Unit B	1,5	4,6	MEP(B,1) = <1,2,4> MEP(B,2) = <5,6> MEP(B,3) = <1,2,3,4,5,6>
Unit C	1	5	MEP(C,1) = <1,3,4,5> MEP(C,2) = <1,2,4,5>

MM-path details

3. Function Testing

- Function testing is defined as "the process of attempting to detect discrepancies between the functional specifications of software and its actual behavior".
- When an integrated system is tested, all its specified functions and external interfaces are tested on the software.
- Every functionality of the system specified in the functions is tested according to its external specifications.
- The function test must determine if each component or business event:
    1. Performs in accordance to the specifications,
    2. Responds correctly to all conditions that may be presented by incoming events / data,
    3. Moves data correctly from one business event to the next
    4. Business events initiated in the order required to meet the business objectives of the system.
- An effective function test cycle must have a defined set of processes and deliverables.

- The primary processes / deliverables for requirements based function test are:

Test planning

- During planning, the test leader with assistance from the test team defines the scope, schedule, and deliverables for the function test cycle.
- He delivers a test plan (document) and a test schedule (work plan)—these often undergo several revisions during the testing cycle.

Partitioning/functional decomposition

- Functional decomposition of a system is the breakdown of a system into its functional components or functional areas.

Requirement definition

- The testing organization needs specified requirements in the form of proper documents to proceed with the function test.

Test case design

- A tester designs and implements a test case to validate that the product performs in accordance with the requirements.

Traceability matrix formation

- Test cases need to be traced / mapped back to the appropriate requirement. A function coverage matrix is prepared.
- This matrix is a table, listing specific functions to be tested, the priority for testing each function, and test cases that contain tests for each function

Functions/Features	Priority	Test Cases
F1	3	T2,T4,T6
F2	1	T1, T3,T5

Function coverage matrix

Test case execution

- As in all the phases of testing, an appropriate set of test cases need to be executed and the results of those test cases recorded.

4.System Testing

- System testing is the type of testing to check the behaviour of a complete and fully integrated software product based on the software requirements specification (SRS) document.
- The main focus of this testing is to evaluate Business / Functional / End-user requirements.

- This is black box type of testing where external working of the software is evaluated with the help of requirement documents & it is totally based on Users point of view.
- For this type of testing do not required knowledge of internal design or structure or code.
- This testing is to be carried out only after System Integration Testing is completed where both Functional & Non-Functional requirements are verified.
- In the integration testing testers are concentrated on finding bugs/defects on integrated modules. But in the Software System Testing testers are concentrated on finding bugs/defects based on software application behavior, software design and expectation of end user.

4.1 Categories of System Testing

1. Recovery Testing

- Recovery is just like the exception handling feature of a programming language. It is a type of non-functional testing.
- Recovery testing is done in order to check how fast and better the application can recover after it has gone through any type of crash or hardware failure etc.
- Recovery testing is the forced failure of the software in a variety of ways to verify that recovery is properly performed.
- Thus Recovery Testing is "the activity of testing how well the software is able to recover from crashes, hardware failures, and other similar problems".

Some examples of Recovery testing are:

- When an application is receiving data from a network, unplug the connecting cable. After some time, plug the cable back in and analyze the application's ability to continue receiving data from the point at which the network connection was broken.
- Restart the system while a browser has a definite number of sessions and check whether the browser is able to recover all of them or not.

"Biezer" proposes that testers should work on the following areas during recovery testing:

- **Restart:** Testers must ensure that al transactions have been reconstructed correctly devices are in proper states.
- **Switchover:** Recovery can also be done if there are standby components and in case of failure of one component, the standby takes over the control.

Security Testing

- It is a type of non-functional testing.
- Security testing is basically a type of software testing that's done to check whether the application or the product is secured or not.

- It checks to see if the application is vulnerable to attacks, if anyone hack the system or login to the application without any authorization.
- It is a process to determine that an information system protects data and maintains functionality as intended.
- The security testing is performed to check whether there is any information leakage in the sense by encrypting the application or using wide range of software's and hardware's and firewall etc.
- Software security is about making software behave in the presence of a malicious attack.

Types of Security Requirements:

- Security Requirements should be associated with each functional requirement.
- For example, the log-on requirement in a client-server system must specify the number of retries allowed, the action to be taken if the log-on specify the number of retries allowed, the action to be taken if the log-on.
- In addition to security concerns that are directly related to particular requirements, a software project has security issues that are global in nature.
- For example, a Web application may have a global requirement that all private customer data of any kind is stored in encrypted form in the database.

How to perform security testing:

- Testers must use a risk based approach, grounded in both the systems architectural reality and the attacker's mindset, to gauge software security adequately.
- By identifying risks and potential loss associated with those risks in the system and creating tests driven by those risks, the tester can properly focus on areas of code in which an attack is likely to succeed.

Elements of Security Testing:

- The basic security concepts that need to be covered by security testing are:

Confidentiality

- A security measure which protects against the disclosure of information to parties other than the intended recipient.

Integrity

- A measure intended to allow the receiver to determine that the information which it receives has not been altered in transit or by anyone other than the originator of the information.

Authentication

- It allows the receiver to have confidence that the information it receives originates from a specifi c known source.

Authorization

- It is the process of determining that a requester is allowed to receive a service or perform an operation.

Availability

- It assures that the information and communication services will be ready for use when expected.
- Information must be kept available for authorized persons when they need it.

Non-repudiation

- A measure intended to prevent the later denial that an action happened, or a communication took place, etc.

2. Performance Testing

- Software performance testing is a means of quality assurance (QA)
- It involves testing software applications to ensure they will perform well under their expected workload.
- Features and Functionality supported by a software system is not the only concern. A software application's performance like its response time, do matter.
- The goal of performance testing is not to find bugs but to eliminate performance bottlenecks.
- Performance testing is done to provide stakeholders with information about their application regarding speed, stability and scalability. But performance testing uncovers what needs to be improved before the product goes to market.
- Without performance testing, software is likely to suffer from issues such as: running slow while several users use it simultaneously, inconsistencies across different operating systems and poor usability.
- Performance testing will determine whether or not their software meets speed, scalability and stability requirements under expected workloads.
- Applications sent to market with poor performance metrics due to non existent or poor performance testing are likely to gain a bad reputation and fail to meet expected sales goals.
- Also, mission critical applications like space launch programs or life saving medical equipments should be performance tested to ensure that they run for a long period of time without deviations.

The following tasks must be done for this thing:

- Develop high level plan including requirements, resources, timeliness, and milestones.
- Develop a detailed performance test plan.
- Specify test data needed.

3. Load Testing

- Load testing is a type of non-functional testing.

- A load test is type of software testing which is conducted to understand the behaviour of the application under a specific expected load for both normal and at peak conditions.
- It helps to identify the maximum operating capacity of an application as well as any bottlenecks and determine which element is causing degradation. E.g. If the number of users are increased then how much CPU, memory will be consumed, what is the network and bandwidth response time.
- Load testing can be done under controlled lab conditions to compare the capabilities of different systems or to accurately measure the capabilities of a single system. It helps you determine how your application behaves when multiple users hits it simultaneously.
- The primary goal of load testing is to define the maximum amount of work a system can handle without significant performance degradation.

Examples of load testing include:

- Downloading a series of large files from the internet.
- Running multiple applications on a computer or server simultaneously.
- Assigning many jobs to a printer in a queue.
- Writing and reading data to and from a hard disk continuously

4. Stress Testing

- It is a type of non-functional testing.
- It involves testing beyond normal operational capacity, often to a breaking point, in order to observe the results.
- It is a form of software testing that is used to determine the stability of a given system.
- It puts greater emphasis on robustness, availability, and error handling under a heavy load, rather than on what would be considered correct behaviour under normal circumstances.
- The goals of such tests ensure that the software does not crash in conditions of insufficient computational resources (such as memory or disk space).
- Thus “Stress Testing tries to break the system under test by overwhelming its resources in order to find the circumstances under which it will crash”.
- The areas that may be stressed in a system are: Input Transactions, Disk Space, Output, Communications, Interaction with users.

5. Usability Testing

- Usability testing is an essential element of quality assurance.
- It is the measure of a product’s potential to accomplish the goals of the user.

- Usability testing is a method by which users of a product are asked to perform certain tasks in an effort to measure the product's ease-of-use, task time, and the user's perception of the experience.
- This look as a unique usability practice because it provides direct input on how real users use the system.
- Usability testing measures human-usable products to fulfil the users purpose.
- The item which takes benefit from usability testing are web sites or web applications, documents, computer interfaces, consumer products, and devices.

What the user wants or excepts from the system can be determined using several ways like:

Area experts

- The usability problems or expectations can be best understood by the subject or area experts who have worked for years in the same area.
- They analyse the system's specific requirements from the user's perspective and provide valuable suggestions.

Group meetings

- Group meeting elicit usability requirements from the user. These meetings result in potential customers' comments on what they would like to see in an interface.

Surveys

- Surveys are another medium to interact with the user. It can also yield valuable information about how potential customers would use a software product to accomplish their tasks.

Analyse similar products

- We can also analyse the experiences in similar kinds of projects done previously and use it in the current project.
- This study will also give us clues regarding usability requirements.

Usability characteristics against which testing are conducted are:

Ease of use

- The users enter, navigate, and exit with relative ease.
- Each user interface must be fit to the intelligence, educational background of the end-user.

Interface steps

- User interface steps should not be misleading.
- The steps should also not be very complex to understand either.

Response time

- The time taken in responding to the user should not be so high that the user is frustrated or will move to some other option in the interface.

Help system

- A good user interface provides help to the user at every step.
- The help documentation should not be redundant; it should be very precise and easily understood by every user of the system.

Error messages

- For every exception in the system, there must be an error message in text form so that users can understand what has happened in the system.
- Error messages should be clear and meaningful.

6. Compatibility/Conversion/Configuration Testing

- Compatibility is a non- functional testing to ensure customer satisfaction.
- It is to determine whether your software application or product is proficient enough to run different browsers, database, hardware, operating system, mobile devices and networks.
- Application could also impact due to different versions, resolution, internet speed and configuration etc. Hence it's important to test the application in all possible manners to reduce failures and overcome from embarrassments of bug's leakage.
- Compatibility test should always perform on real environment instead of virtual environment.
- Test the compatibility of application with different browsers and operating systems to guarantee 100% coverage.

Types of Software compatibility testing:

Operating systems

- The specifications must state all the targeted end-user operating systems on which the system being developed will be run.

Software/Hardware

- The product may need to operate with certain versions of Web browsers, with hardware devices such as printers, or with other software's such as virus scanners or word processors.

Conversion testing

- Compatibility may also extend to upgrades from previous versions of the software.
- Therefore, in this case, the system must be upgraded properly and all the data and information from the previous version should also be considered.

Ranking of possible configurations

- There will be a large set of possible configurations and compatibility concerns, the testers must rank the possible configurations in order, from the most to the least common, for the target system.

Identification of test cases

- Testers must identify appropriate test cases and data for compatibility testing.
- It is usually not feasible to run the entire set of possible test cases on every possible configuration, because this would take too much time.
- Therefore, it is necessary to select the most representative set of test cases that confirms the application's proper functioning on a particular platform.

5.Acceptance Testing

- After the system test has corrected all or most defects, the system will be delivered to the user or customer for acceptance testing.
- Acceptance testing is basically done by the user or customer although other stakeholders may be involved as well.
- The goal of acceptance testing is to establish confidence in the system.
- Acceptance testing is most often focused on a validation type testing.
- Thus "*Acceptance Testing is the formal testing conducted to determine whether a software system satisfies its acceptance criteria and to enable buyer to determine whether to accept the system or not.*"
- Thus acceptance testing is designed to:
    - Determine whether the software is fit for the user to use.
    - Making users confident about product
    - Determine whether a software system satisfies its acceptance criteria.
    - Enable the buyer to determine whether to accept the system.

Types of Acceptance Testing

1. Alpha Testing
2. Beta Testing

Alpha Testing

- Alpha is the test period during which the product is complete and usable in a test environment, but not necessarily bug-free.
- It is the final chance to get verification from the customers made in the final development stage.
- Therefore, alpha testing is typically done for two reasons:
    - to give confidence that the software is in a suitable state to be seen by the customers.
    - to find bugs that may only be found under operational conditions. Any other major defects or performance issues should be discovered in this stage.

- Alpha testing is performed at the development site, testers and users together perform this testing, if any problem comes up, it can be managed by the testing team.

Entry Criteria for Alpha

- All features are complete/testable (no urgent bugs).
- High bugs on primary platforms are fixed/verified.
- 50% of medium bugs on primary platforms are fixed/verified.
- Performance has been measured/compared with previous releases.
- Usability testing and feedback
- Alpha sites are ready for installation.

Exit Criteria from Alpha

After alpha testing, we must:

- Get responses/feedbacks from customers.
- Prepare a report of any serious bugs being noticed.
- Prepare a report of any serious bugs being noticed.

Beta Testing

- Beta is the test period during which the product should be complete and usable in a production environment.
- The purpose of the beta testing is to test the company's ability to deliver and support the product.
- It is also known as field testing. It takes place at customer's site. It sends the system to users who install it and use it under real-world working conditions.
- Beta testing can be considered "pre-release testing."
- Beta testing is also sometimes referred to as user acceptance testing (UAT) or end user testing.
- The experiences of the early users are forwarded back to the developers who make final changes before releasing the software commercially.

Entry Criteria for Beta

- Positive responses from alpha site.
- Customer bugs in alpha testing have been addressed.
- There are no fatal errors which can affect the functionality of the software.
- Beta sites are ready for installation.

Exit criteria to Beta

- Get response / feedbacks from the beta testers.
- Prepare a report of all serious bugs.
- Notify bug–fixing issues to developers.

Regression testing

Regression testing: Progressives Vs regressive testing, Regression testability, Objectives of regression testing, When regression testing done?, Regression testing types, Regression testing techniques.

1.Progressives Vs regressive testing

- All the test case design methods or testing technique, discussed till now are referred to as progressive testing or development testing.
- The purpose of regression testing is to confirm that a program or code change has not adversely affected existing features.
- Regression testing is nothing but full or partial selection of already executed test cases which are re-executed to ensure existing functionalities work fine.
- This testing is done to make sure that new code changes should not have side effects on the existing functionalities.
- It ensures that old code still works once the new code changes are done.

Need of Regression Testing:

- Change in requirements and code is modified according to the requirement.
- New feature is added to the software
- Defect fixing
- Performance issue fix

Definition:

Regression testing is the selective retesting of a system or component to verify that modifications have not caused unintended effects and that the system or component still complies with its specified re

2 Regression Testing Produces Quality Software Requirements.

- This changed software requires to be fully tested again so as to ensure:

(a) bug-fixing has been carried out successfully

(b) the modifications have not affected other unchanged modules

(c) the new requirements have been incorporated correctly.

- This process of executing the whole set of test again and again may be time-consuming and frustrating, but it increases the quality of software. Therefore, the creation, maintenance, and execution of a regression test suite helps to retain the quality of the software.
- The importance of regression testing is well-understood for the following reasons:
    1. It validates the parts of the software where changes occur.
    2. It validates the parts of the software which may be affected by some changes, but otherwise unrelated.

3. It ensures proper functioning of the software, as it was before changes occurred.
4. It enhances the quality of software, as it reduces the risk of high-risk bugs.

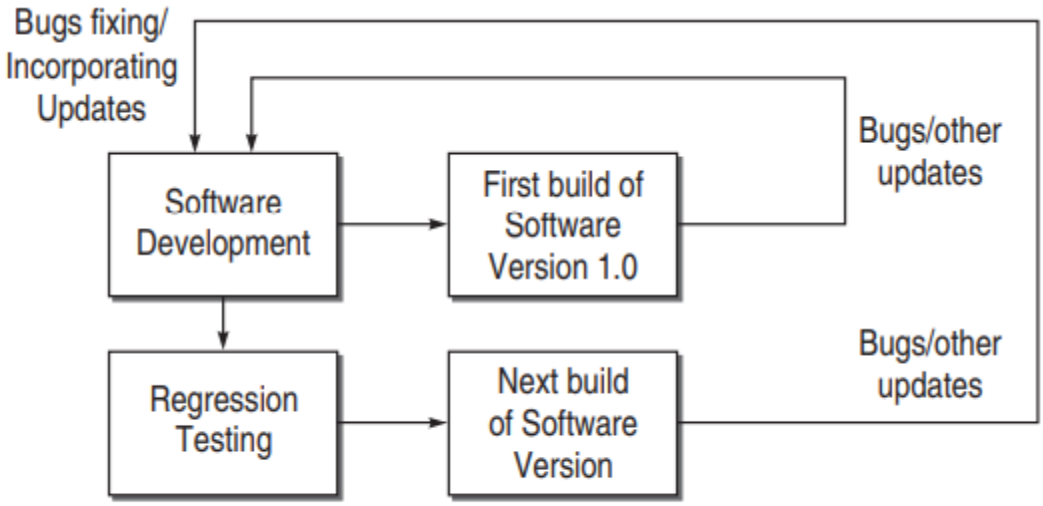


Figure 8.1 Regression testing produces Quality Software

3.Regression Testability

- Regression testability refers to the property of a program, modification or test suite that lets it be effectively and efficiently regression-tested.
- We can classify a program as regression testable if most single statement modifications to the program entail(involves) rerunning a small proportion of the current test suite.
- If regression testability is considered at an early stage of development, it can provide significant (sufficiently great) savings in the cost of development and maintenance of the software.

4.Objectives of Regression Testing

It tests to check that the bug has been addressed

- The first objective in bug fix testing is to check whether the bug-fixing has worked or not.

- Therefore, if run exactly the same test that was executed when the problem was first found. If the program fails on this testing, it means the bug has not been fixed correctly and there is no need to do any regression testing further.

It finds other related bugs

- The developer has fixed only the symptoms of the reported bugs without fi xing the underlying bug.
- Regression tests are necessary to validate that the system does not have any related bugs.

It tests to check the effect on other parts of the program

- It may be possible that bug-fixing has unwanted consequences on other parts of a program.
- Therefore, it is necessary to check the influence of changes in one part or other parts of the program.

5.When regression testing done?

Software Maintenance:

1. **Corrective Maintenance:** Changes made to correct a system after a failure has been observed.
2. **Adaptive Maintenance:** Changes made to achieve continuing compatibility with the target environment or other systems.
3. **Perfective Maintenance:** Changes made to improve or add capabilities.
4. **Preventive Maintenance:** *C*hanges made to increase robustness, maintainability, portability, and other features.
5. **Rapid Iterative Development**: The extreme programming approach requires that a test be developed for each class and that this test be re-run every time the class changes.
6. **Compatability Assesment and Benchmarking**:Some test suites designed to be run on a wide range of platforms and applications to establish conformance with a standard or to evaluate time and space performance.

6.Regression Testing Types

Bug-fix Regression: This testing is performed after a bug has been reported and fixed.

Side-Effect Regression / Stability Regression:

- It involves retesting a substantial (important) part of the product.
- The goal is to prove that the changes has no detrimental effect on something that was earlier in order.

7 .Defining Regression Test Problem

P denotes a program or procedure,
P' denotes a modified version of P,
S denotes the specification for program P,
S' denotes the specification for program P',
$P(i)$ refer to the output of P on input i,

$P'(i)$ refer to the output of P' on input i, and
$T = \{t_1, \ldots. t_n\}$ denotes a test suite or test set for P.

1.IS REGRESSION TESTING A PROBLEM?

Regression testing is considered a problem, as the existing test suite with probable additional test cases needs to be tested again and again whenever there is a modification.

The following difficulties occur in retesting:

- Large systems can take a long time to retest.
- It can be difficult and time-consuming to create the tests.
- It can be difficult and time-consuming to evaluate the tests. Sometimes, it requires a person in the loop to create and evaluate the results.
- Cost of testing can reduce resources available for software improvements.

3. REGRESSION TESTING PROBLEM

Given a program P, its modified version P', and a test set T that was used earlier to test P, find a way to utilize T to gain sufficient confidence in the correctness of P'.

8.Regression Testing Techniques

There are different techniques for regression testing. They are:

Regression test selection technique:

- This technique attempts to reduce the time required to retest a modified program by selecting some subset of the existing test suite.

Test case prioritization technique:

- Regression test prioritization attempts to reorder a regression test suite so that those tests with the highest priority according to some established criteria, are executed earlier in the regression testing process rather than those lower priority. There are two types of prioritization:

(a) General Test Case Prioritization:

- For a given program P and test suits T, we prioritize the test cases in T that will be useful over a succession of subsequent modified versions of P, without any knowledge of the modified version.

(b) Version-Specific Test case Prioritization:

- We prioritize the test cases in T, when P is modified to P', with the knowledgeof the changes made in P.

Test Suite Reduction Technique:

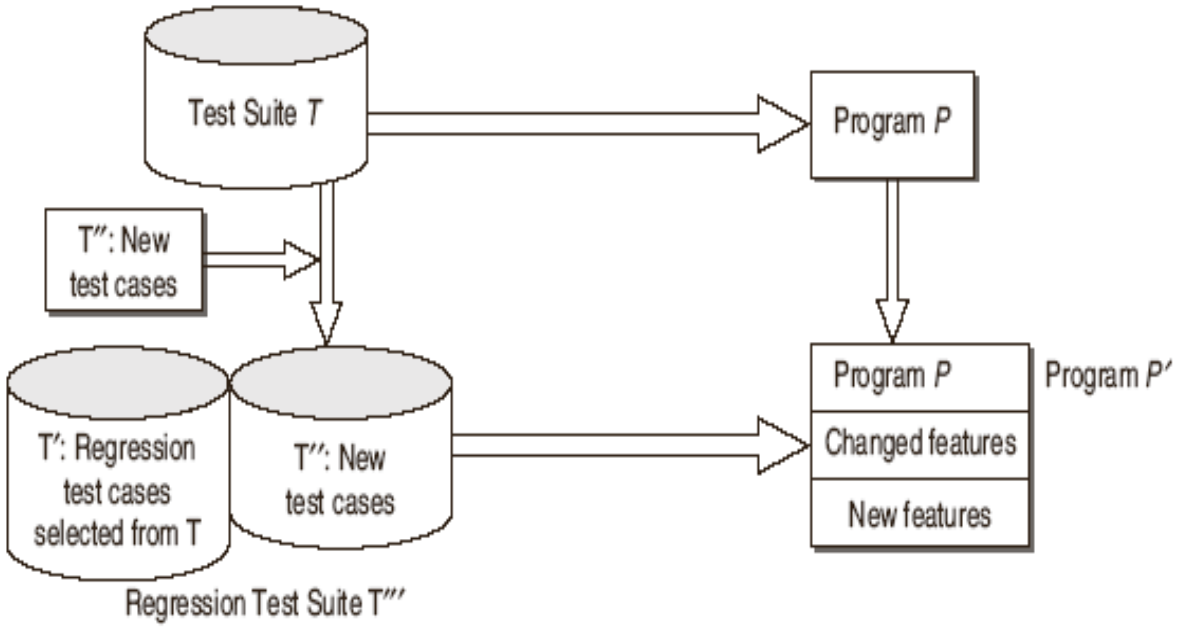
- It reduces testing costs by permanently eliminating redundant test cases form test suites in terms of codes of functionalities exercised.

1.Selective Retest Technique

- Selective retest technique attempts to reduce the cost of testing by identifying the portions of P' (modified version of Program) that must be exercised but the regression test suite.
- Following are the characteristic features of the selective retest technique:
    - It minimizes the resources required to regression test a new version.
    - It is achieved by minimizing the number of test cases applied to the new version.
    - It analyses the relationship between the test cases and the software elements they cover.
    - It uses the information about changes to select test cases.

Steps in Selective retest technique:

1. Select T' subset of T, a set of test cases to execute on P'.
2. Test P' with T', establishing correctness of P' with respect to T'.
3. If necessary, create T'‖, a set of new functional test cases for P'.
4. Test P' with T', establishing correctness of P' with respect to T".
5. Create T"'. a new test suite and test execution profile for P', from T, T' and T".



Selective Retest Technique

Strategy for Test Case Selection

- For large software systems, there may be thousands of test cases available in its test suite.
- When a change is introduced into the system for next version, rerunning all the test cases is a costly and time consuming task.
- Therefore a need for selecting a subset of test cases from the original test suite is necessary.
- So, an effective test case selection strategy is to be designed based on the code coverage.

Selection criteria based on Code

- The code-based testing is that potential failures can only be detected if the parts of code that can cause faults are executed.
- All the code based regression test selection techniques attempt to select a subset T ' of T that will be helpful in establishing confidence that *P* ¢ *s* functionality has been required.
- All code-based test selection techniques are concerned with locating tests in *T* that expose faults in P'.

The following tests are based on these criteria.

Fault revealing test cases: A test case *t* detects a fault in P' if it causes P' to fail. Hence *t* is fault-revealing for P'.

Modification revealing Test cases: A test case *t* is modification-revealing for *P* and P' if and only if it causes the outputs of *P* and P' to differ.

Modification traversing Test cases: A test case *t* is modification-traversing if and only if it executes new or modified code in P'.

Regression Test Selection Techniques

Minimization Techniques:

- Minimization-based regression test selection techniques attempt to select minimal sets of test cases from T that yield coverage of modified portions of P.
- The technique uses a 0-1 integer programming algorithm to identify a subset T' of T that ensures that every segment is a modified segment by at least one test case in T9.

Dataflow Techniques:

- Dataflow-coverage-based regression test selection techniques select test cases that exercise data interactions that have been affected by modifications.
- For example, the technique requires that every definition that is deleted from P, or modified for P' be tested.
- The technique selects every test case in T that, when executed on P, exercised deleted or modified definition or executed a statement containing a modified.

Safe Techniques:

- Most regression test selection techniques—minimization and dataflow techniques are not designed to be safe.
- Techniques that are not safe can fail to select a test case that would have revealed a fault in the modified program.
- Safe regression test selection techniques guarantee that the selected subset, T', contains all test cases in the original test suite T that can reveal faults in P'.

Ad Hoc/Random Techniques:

- When time constraints prohibit the use of a retest, no test selection tool is available, developers often select test cases based on ―hunches,‖ or loose associations of test cases with functionality.
- Another simple approach is to randomly select a number of test cases from T.

Retest-All Technique:

- The retest-all technique simply reuses all existing test cases.
- To test P', the technique effectively ―selects‖ all test cases in T.

Evaluating Regression Test Selection Technique

To choose a technique for practical application, the following are the categories for evaluating the regression test selection techniques:

Inclusiveness: Let M be a regression test selection technique. Inclusiveness measures the extent to which M chooses modification revealing tests from T for inclusion in T', where *M* is a regression test selection technique.

Definition

Suppose T contains n tests that are modification revealing for P and P', and suppose M selects m of these tests. The inclusiveness of M relative to P, P', and T is:

1. INCL(*M*) = (100 *(*m*/*n*)%, if $n \neq 0$
2. INCL(*M*)% = 100%, if $n = 0$

For example, if *T* contains 100 tests of which 20 are modification-revealing for *P* and P', and *M* selects 4 of these 20 tests, then *M* is 20% inclusive relative to *P*, P', and *T*. If *T* contains no modification-revealing tests, then every test selection technique is 100% inclusive relative to *P*, P', and *T*.

Precision: Let M be a regression test selection technique. Precision measures the extent to which M omits tests that are non modification-revealing. We define precision relative to a particular program, modified program, and test suite, as follows:

Definition

Suppose T contains n tests that are non modification-revealing for P and P' and suppose M omits m of these tests. The precision of M relative to P, P: and T is

Precision = 100 *(*m*/*n*) % if $n \neq 0$

Precision = 100 % if $n = 0$

For example, if T contains 50 tests of which 44 are non modification-revealing for P and P', and M omits 33 of these 44 tests, then M is 75% precise relative to P, P', and T. If T contains no non- modification revealing tests, then every test selection technique is 100% precise relative to P, P', and T.

Efficiency:

- We measure the efficiency of regression test selection techniques in terms of their space and time requirements.

- Where time is concerned, a test selection technique is more economical than the retest-all technique if the cost of selecting T' is less than the cost of running the tests in T-T' Space efficiency depends on the test history and program analysis must store.
- Thus, both space and time efficiency depend on the size of the test suite that a technique selects, and on the computational cost of that technique.

2.Regression Test Prioritization:

- The regression test prioritization approach is different as compared to selective retest techniques.
- Regression test prioritization attempts to reorder a regression test suite so that those tests with the highest priority are executed earlier in the regression testing process than those with a lower priority.

The steps for this approach are:

1. Select T ' from T, a set of test cases to execute on P'.
2. Produce T '$_P$, a permutation of T ', such that T '$_P$ will have a better rate of fault detection than T '.
3. Test P' with T '$_P$ in order to establish the correctness of P' wrt T '$_P$.
4. If necessary create T ", a set of new functional or structural tests for P'.
5. Test P' with T " in order to establish the correctness of P' wrt T ".
6. Create T "', a new test suite for P', from T, T '$_P$ and T".

MULTIPLE CHOICE QUESTIONS

1. A changed version that has not passed a regression test is called _____(b) _____.

(a) baseline version (b) delta version (c) delta build (d) none of the above

2. Regression testability is a function of ___(a) _____.

(a) design of the program and the test suite (b) design of the program (c) test suite (d) none of the above

3. Regression number is computed using information about the __(b) _____.

(a) design coverage of the program (b) test suite coverage of the program (c) number of faults (d) all of the above

4. Changes made to correct a system after a failure has been observed is called ____(c)_____.

(a) perfective maintenance (b) adaptive maintenance (c) corrective maintenance (d) all of the above

5. Changes made to achieve continuing compatibility with the target environment or other systems is called _____(b) ____.

(a) perfective maintenance (b) adaptive maintenance (c) corrective maintenance (d) all of the above

6. Changes designed to improve or add capabilities is called _____(a) ____.

(a) perfective maintenance (b) adaptive maintenance (c) corrective maintenance (d) all of the above

7. Changes made to increase robustness, maintainability, portability, and other features is called _____(b) ___.

(a) perfective maintenance (b) preventive maintenance (c) adaptive maintenance (d) corrective maintenance

8. Which statement best characterizes the regression test selection? **Answer:** (a) & (b)

(a) Minimize the resources required to regression test a new version (b) Typically achieved by minimizing the number of test cases applied to the new version (c) Select a test case which has caused the problem (d) None of the above

9. Problem to select a subset T' of T with which P' will be tested is called ___(c) _____.

(a) coverage identification problem (b) test suite execution problem (c) regression test selection problem (d) test suite maintenance problem

10. The problem to identify portions of P' or its specification that requires additional testing is called _____(a) __.

(a) coverage identification problem (b) test suite execution problem (c) regression test selection problem (d) test suite maintenance problem

11. The problem to execute test suites efficiently and checking test results for correctness is called _____(b) __.

(a) coverage identification problem (b) test suite execution problem (c) regression test selection problem (d) test suite maintenance problem

12. The problem to update and store test information is called ___(d) _____.

(a) coverage identification problem (b) test suite execution problem (c) regression test selection problem (d) test suite maintenance problem

13. A test case t is modification-revealing for P and P' if ___(c) _____.

(a) it executes new or modified code in P' (b) it detect a fault in P' if it causes P' to fail (c) it causes the outputs of P and P' to differ (d) all of the above

14. A test case t is modification-traversing if _____(a) _____.

(a) it executes new or modified code in P' (b) it detect a fault in P' if it causes P' to fail (c) it causes the outputs of P and P' to differ (d) all of the above

15. A test case t is fault-revealing if ____ (b) _____.

(a) it executes new or modified code in P' (b) it detect a fault in P' if it causes P' to fail (c) it causes the outputs of P and P' to differ (d) all of the above

16. Regression testing is helpful in ___(d)_____. (a) detecting bugs (b) detecting undesirable side effects by changing the operating environment (c) integration testing (d) all of the above

1. (b) 2. (a) 3. (b) 4. (c) 5. (b) 6. (a) 7. (b) 8. (a) & (b) 9. (c) 10. (a) 11. (b) 12. (d) 13. (c) 14. (a) 15. (b) 16. (d)